

LeNet-5

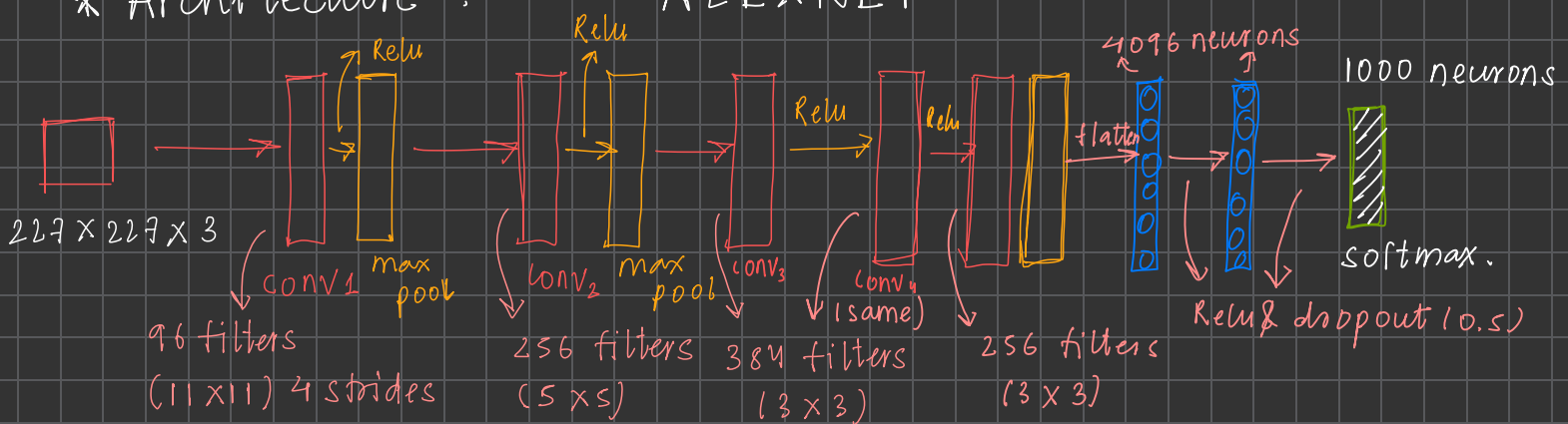
- ⇒ handwritten dataset. (MNIST dataset)
- ⇒ $32 \times 32 \times 1$ (grayscale)
- ⇒ Tanh or sigmoid activation function.
- ⇒ Average pool.
- ⇒ Total parameters: 60,000
- ⇒ Architecture: 2 conv. layer + 3 fully connected

AlexNet

- ⇒ real world objects 1000 categories (Image Net)
- ⇒ $227 \times 227 \times 3$ (RGB)
- ⇒ ReLU is used.
- ⇒ Max pooling.
- ⇒ Around 60 millions parameters
- ⇒ Architecture: 5 conv. + 3 fully connected

* Architecture :-

ALEXNET



* Drawbacks :-

- takes lots of memory to train. (60 million params.)
- Inefficient architecture.

AlexNet

⇒ 8 layers (5 conv. + 3 fully connected)

⇒ Input image :
 $227 \times 227 \times 3$

⇒ Mixed filter sizes -
 11×11 , 5×5 , 3×3

⇒ ~ 61 million parameters.

VGGNet-16

visual geometric group.

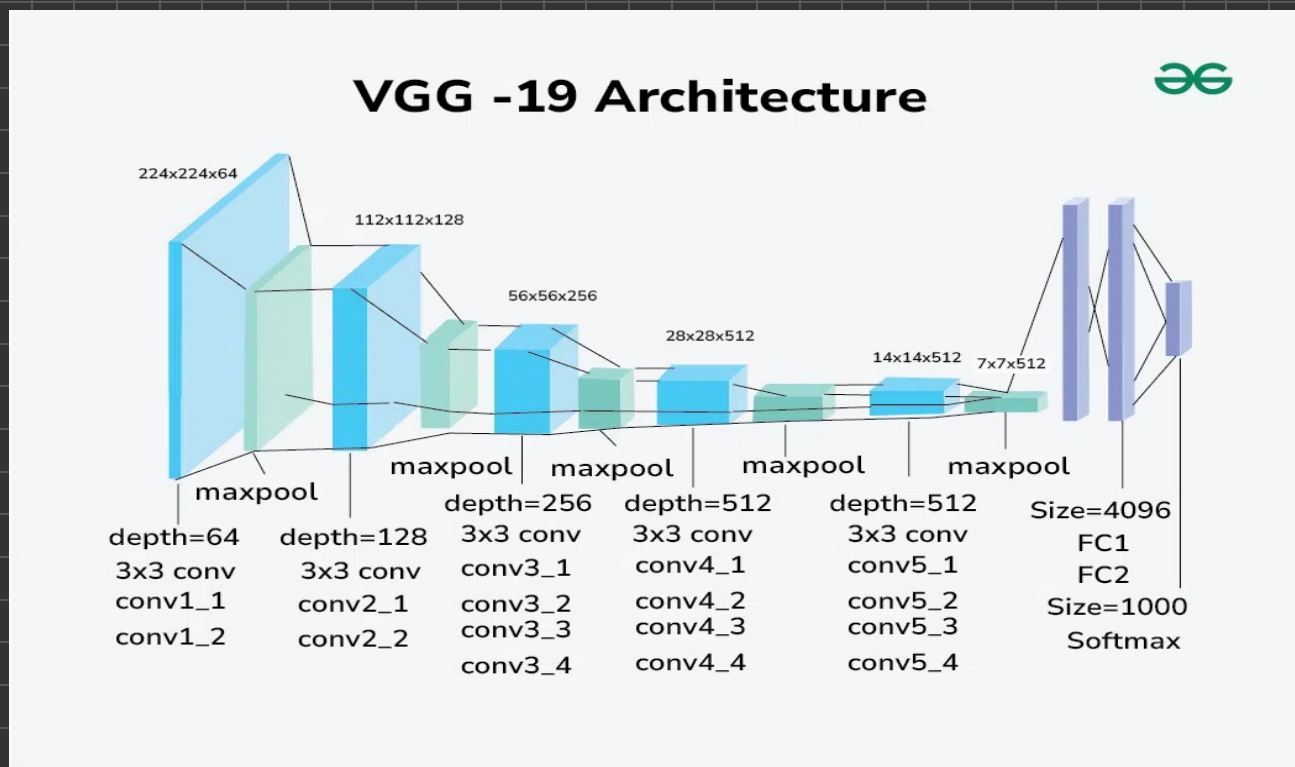
⇒ 16 layers (13 conv. + 3 fully connected)

⇒ Input image :
 $224 \times 224 \times 3$

⇒ Uniform filters i.e.
 3×3 .

⇒ ~ 138 million parameters.

* 3×3 filter in VGGNet : instead of heavy 7×7 , use 3×3 all over remains exactly 7×7 but we have to ReLU 3 - times instead of 1.



VGGNet

- ⇒ stacks 3×3 layers sequentially.
- ⇒ 16 - layers.
- ⇒ ~ 138 mill. parameters.
- ⇒ End layer :- massive fully connected layer.

ResNet

- ⇒ use skip connections to bypass layers.
- ⇒ 50 layers.
- ⇒ ~ 26 mill. parameters.
- ⇒ End layer :- Global avg. pooling (saves param.)

* Skip connections :- Normally layer tries to learn direct mapping $H(x)$. But ResNet asks layer to learn difference $f(x)$. So it can be added into original output $F(x) + x$.

* Solved vanishing gradient problem.

In vanishing gradient problem, shallow layers stops learning & stuck at suboptimal state.

"Its not overfitting, its undertraining model can't learn due to bad optimization".

"Layer doesn't learn full transformation but only learns the residual".

$$H(x) = f(x) + x$$

$H(x)$ → desired output
 $f(x)$ → residual
 x → input

Blurred
input image



Skip connection

residual

Skip connection

High-res
output image



Green line w/o any change preserves basic structure of image.

residual: it's like missing details - sharp edges, fine textures, high-frequency details.

Resnet is similar to euler method (if step = 1)

ResNet

skip connections - (shortcut)

Addition ($x + F(x)$)

Stays same in a block.

High parameters.

Good information flow.

DenseNet

Dense connections (every connected to everyone)

Concatenation ($[x_0, x_1, \dots, x_n]$)

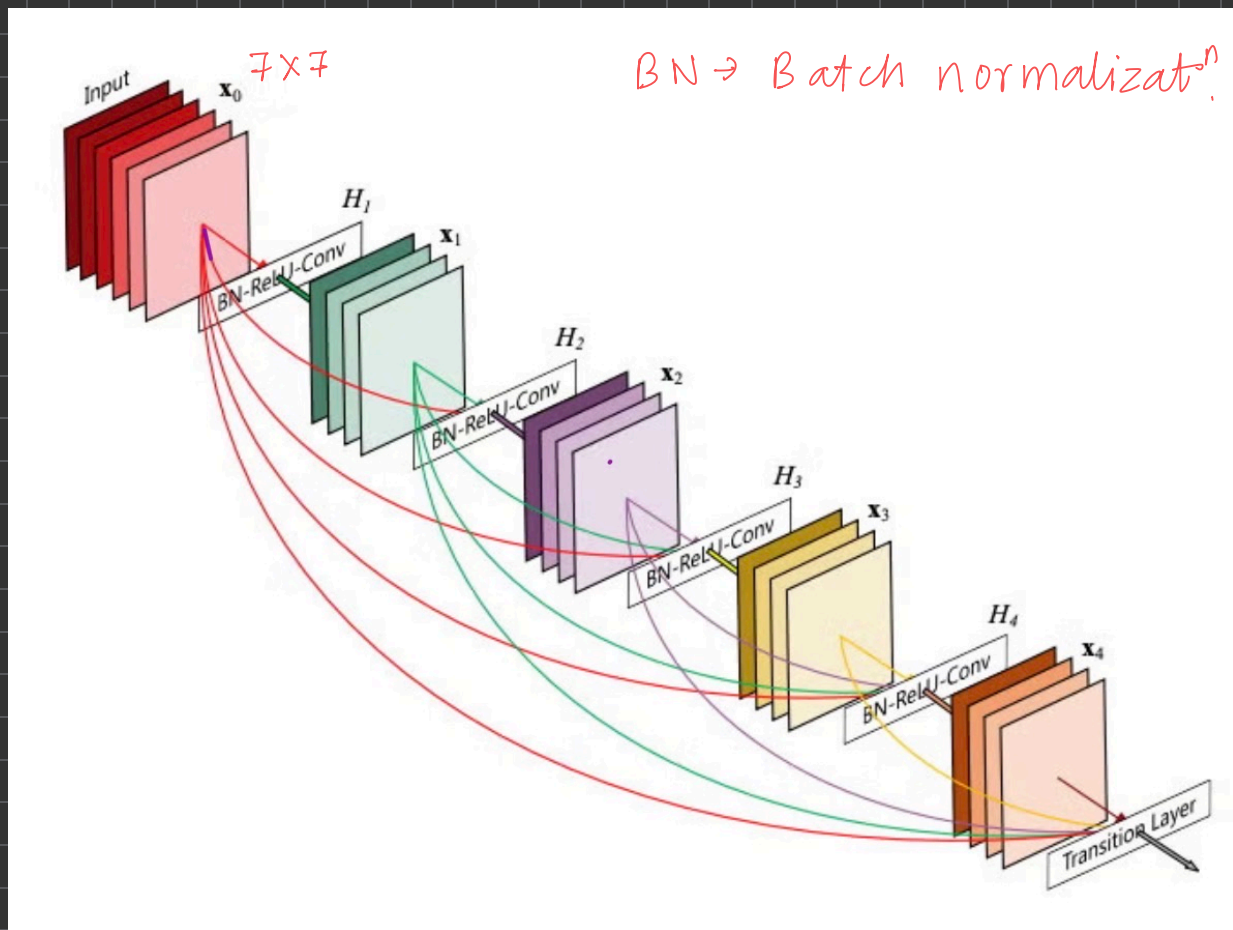
Becomes thicker after every layer.

Low parameters.

Maximum (direct access to input).

* Feature reuse - As all previous layers concatenate feature maps, network do not have to learn same feature again n again.

* Growth rate - (K) each layer produces 32 filters only. As every layer connects output with it.



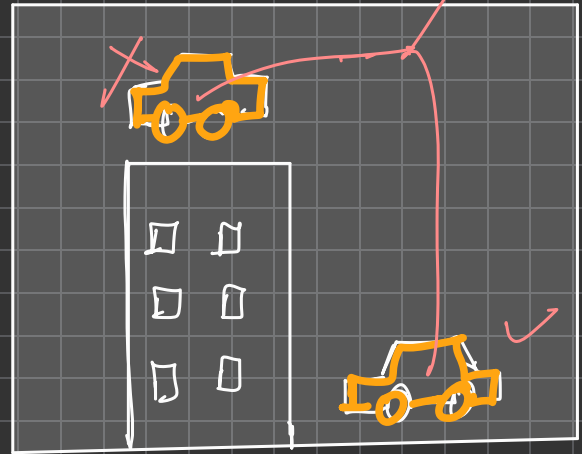
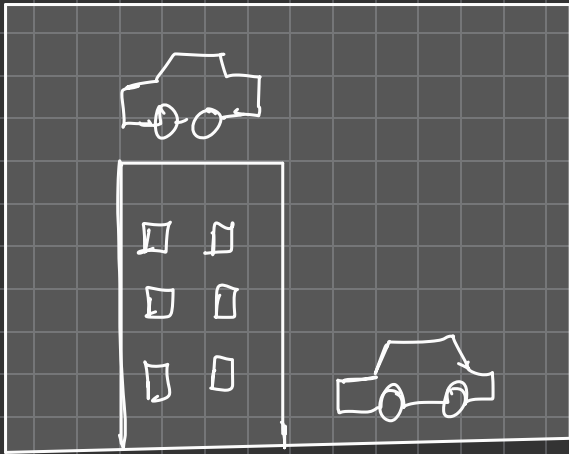
Drawbacks of CNN

Locality
(small filters
 3×3)

Translation
invariance

Weight
sharing
(reuse similar
filter)

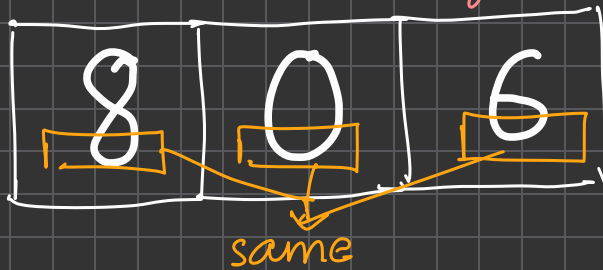
Traced but
unethical.



⇒ Struggle in global context.

⇒ Stack more layers i.e. vanishing gradient.

⇒ Not flexible



Receptive field? How much area you're looking at once.

Transformers :- Embedding → Transformer block
output probabilities

"The cat sat on the —"

→ Tokenization (ignore spaces)

⇒ 5 tokens

⇒ $V = 50257$ # vocabulary size

⇒ $d_m = 768$ # embedding size.

→ token ID's ⇒ [464, 1135, 1132, 319, 262]

→ Each token as vector representation.

→ We're only processing one sentence so batch size is (1, 5)

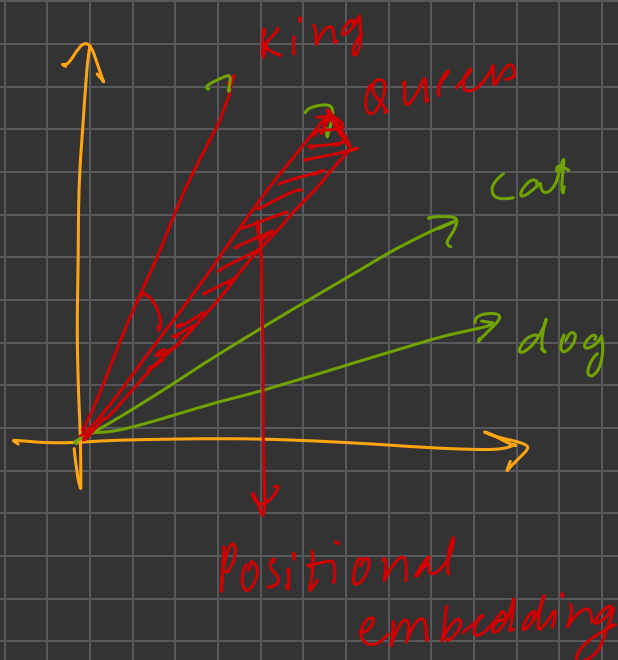
→ Token representation (1, 5, 768)
(positional embedding - 768)

12 attention head.

$768 / 12 \Rightarrow (64)$

Encoder ⇒ Bert

Decoder ⇒ LPT



● Vision Transformers

In NLP to understand the text we use words as tokens in "transformers" (like ChatGPT).

similarly, here we divide image into "patches"

* Why ViT instead of CNN?

CNN works very well on small datasets, but on massive datasets with heavy computational power ViT's works very well.

1. Self-attention mechanism
2. Patch embeddings
3. Positional encodings
4. CLS token.

* Self attention mechanism :-

Its basically asks patches to get meaning, how much attention is required?

Q : query (what I'm looking for?)

K : Key (what is my identity?)

V : Value (What is my actual information?)

> Mathematically,

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_K}} \right) V$$

⇒ Dot product (QK^T) - we multiply one patch query with all patches key. If it matches the value of key & query then dot product will be high (strong relationship)

⇒ scaling ($\sqrt{d_k}$) - In high dimension, large value of dot product results in vanishing gradient. ($\sqrt{d_k}$) makes the no. stable.

⇒ Softmax - get values from 0 to 1

⇒ Multiply by Value (V) :- Actual value (V) is multiplied so that their weight can reflect in final output.

> Advantages

1. Nets all context from first layer unlike CNN (only 3×3)
2. Not have fixed filters, changes based on input data.

> Disadvantages

1. It requires millions of img, as it learns everything from scratch.

* Patch embedding :-

A process in which we do "flattening" & make it vector. This is called Patch embedding.

Original img :- $224 \times 224 \times 3$

Patch create :- (16×16)

$$\text{No. of patches (N)} = \frac{224 \times 224}{16 \times 16} = \underline{\underline{196}}$$

196 patches !!

1 - flatten patch - $16 \times 16 \times 3 = 768$ nos.

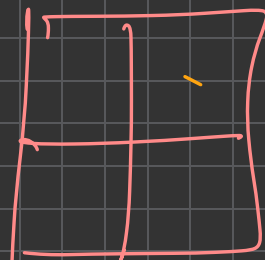
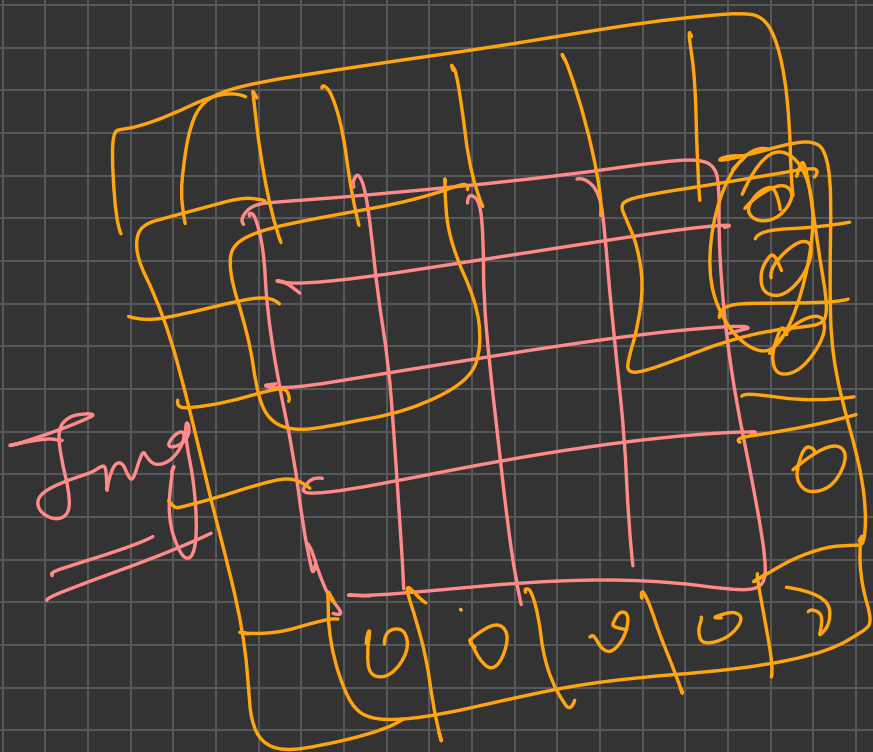
> Advantage - It makes manageable.

> Disadvantage - It becomes isolated (unless attention is applied.)

* Positional encoding :-

Assign value to remember spatial location.

$$E_{\text{final}} = E_{\text{patch}} + P_{\text{position}}$$



kernel

Why we need padding??

[[] [] [] [] ...]

TP	FN
FP	TN

① Image Classification

(Cat vs Dog)
→ Architecture

LeNet, AlexNet, VGG, ResNet, DenseNet

② Image Segmentation → U-Net, ---

③ Object Detection →

↓

Yolo, ResNet →

Bob

④ Image Captioning.

⑤ Image Super-Resolution

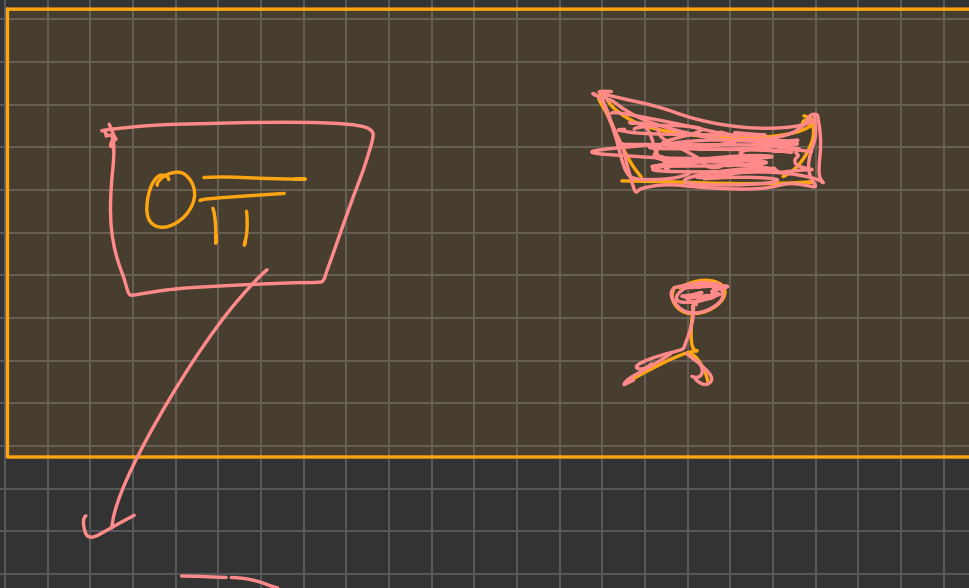
Input: Low Resolution, Output:
H.Q. I

Part 1 + CNN Fundamentals

How Training happens in CNN.

Weights, Filter values etc.

Weight Sharing.



➡ 2-month

(1) Explainable AI

(2) (1) Vision Transformers ✓

Attention Based Networks

Dataset

Skin-Cancer
Detection →

XAI
Grayscale